

1. Shell and Bash Basics

A **shell** is a program that reads your command and executes it. **Bash** is the most common shell on Linux.

Check which shell you are using:

```
echo $SHELL
```

Check the bash version:

```
bash --version
```

Important idea

A **shell script** is just a text file that contains commands you could type in terminal. The shell runs the file line by line.

2. Creating and Running Your First Script

We usually save shell scripts with **.sh** extension (example: **first.sh**).

Create a file using nano editor:

```
nano first.sh
```

Write this code inside the file:

```
#!/bin/bash
# first.sh - my first bash script

echo "Hello, Linux!"
```

Line-by-line explanation

- **#!/bin/bash** is called **shebang**. It tells Linux to use bash to run this script.
- **#** starts a comment. Comments are ignored by bash.
- **echo** prints text on the screen.

Save and exit nano:

```
Ctrl + O  (save)
Enter     (confirm)
Ctrl + X  (exit)
```

Give execute permission and run:

```
chmod +x first.sh
./first.sh
```

Why we use **./**?

Linux does not run files from the current folder automatically. **./** means: run this program from the current directory.

3. Printing Output with echo

echo prints text or variable values.

```
#!/bin/bash
echo "One line"
echo "My name is Ali"
```

Useful echo options:

```
echo -n "No new line at end"
echo -e "Line1
Line2
Line3"    # -e enables special characters like \n
```

Tip: Use quotes when your text has spaces.

4. Variables (Storing Data)

Variables store values. In bash, you do not declare types (no int/float/string keywords).

Rules (students must memorize):

- No space around =
- Use **\$name** to read a variable value
- Use quotes for text values

Example:

```
#!/bin/bash
```

```
name="Ali"
```

```
age=20
```

```
echo "Name: $name"
```

```
echo "Age:  $age"
```

Common mistakes:

```
name = "Ali"    # WRONG (spaces around =)
```

```
echo "Name: name" # WRONG (missing $)
```

5. Taking Input from User (read)

The **read** command waits for the user to type something, then stores it in a variable.

```
#!/bin/bash
```

```
echo "Enter your name:"  
read name
```

```
echo "Enter your age:"  
read age
```

```
echo "Welcome $name, age = $age"
```

Line-by-line idea:

- **read name** stores the input into variable **name**.
- We print variables using **\$name** and **\$age**.

Read with a prompt (single line):

```
read -p "Enter city: " city  
echo "City = $city"
```

6. Quotes and Why They Matter

Bash treats spaces as separators. Quotes help keep words together.

Double quotes allow variables to expand:

```
name="Ali"
echo "Hello $name"    # prints: Hello Ali
```

Single quotes do NOT expand variables:

```
name="Ali"
echo 'Hello $name'    # prints: Hello $name
```

When to use quotes?

- Use double quotes for messages with variables.
- Use quotes when a value contains spaces (file names, sentences).

7. Arithmetic (Math) in Bash

For integer math, bash commonly uses **`$(( ... ))`**.

```
#!/bin/bash
```

```
read -p "Enter a: " a
read -p "Enter b: " b
```

```
sum=$((a + b))
sub=$((a - b))
mul=$((a * b))
```

```
echo "sum = $sum"
echo "sub = $sub"
echo "mul = $mul"
```

Division (integer division):

```
q=$((a / b))    # 7/2 becomes 3
r=$((a % b))    # remainder
```

Note: Bash math is integer by default. For decimals, use `bc` (advanced topic).

8. if-else (Decision Making)

Use **if** when you want to run different commands based on a condition.

Numeric comparison uses special operators like **-gt**, **-ge**, etc.

Operator	Meaning	Example (a=5, b=3)
-eq	equal	[\$a -eq \$b] (false)
-ne	not equal	[\$a -ne \$b] (true)
-gt	greater than	[\$a -gt \$b] (true)
-lt	less than	[\$a -lt \$b] (false)
-ge	greater or equal	[\$a -ge \$b] (true)
-le	less or equal	[\$a -le \$b] (false)

Example: check voting eligibility

```
#!/bin/bash

read -p "Enter age: " age

if [ $age -ge 18 ]
then
    echo "Eligible to vote"
else
    echo "Not eligible"
fi
```

Important rules:

- Spaces inside **[]** are mandatory.
- **[]** is actually a command named **test**.
- End if block using **fi** (if backwards).

9. String Comparison

Strings are text values like names, cities, etc.

Common string tests:

```
[ "$s1" = "$s2" ]      # equal
[ "$s1" != "$s2" ]     # not equal
[ -z "$s" ]            # string length is zero (empty)
[ -n "$s" ]            # string length is not zero
```

Example:

```
#!/bin/bash

read -p "Enter your username: " user

if [ "$user" = "admin" ]
then
    echo "Hello Admin"
else
    echo "Hello $user"
fi
```

Tip: Always quote variables in string comparisons to avoid errors when input is empty.

10. File and Directory Tests (very useful in Linux)

Bash can check if a file exists, if a directory exists, if a file is readable, etc.

Common file test operators:

Test	Meaning
-f file	file exists and is a regular file
-d dir	directory exists
-e path	path exists (file or directory)
-r file	file is readable
-w file	file is writable
-x file	file is executable
-s file	file exists and size > 0

Example: check file exists

```
#!/bin/bash

read -p "Enter filename: " fname

if [ -f "$fname" ]
then
    echo "File exists: $fname"
else
    echo "File not found"
fi
```

11. Loops (Repeating Work)

Loops help you repeat commands.

11.1 for loop

```
#!/bin/bash

for i in 1 2 3 4 5
do
    echo "i = $i"
done
```

Explanation:

- **for i in ...**: variable i takes each value one by one.
- **do** starts the loop body.
- **done** ends the loop.

11.2 while loop

```
#!/bin/bash

i=1
while [ $i -le 5 ]
do
    echo "i = $i"
    i=$((i + 1))
done
```

11.3 until loop (runs until condition becomes true)

```
#!/bin/bash

i=1
until [ $i -gt 5 ]
do
    echo "i = $i"
    i=$((i + 1))
done
```

12. case (Menu-style Decision)

case is cleaner than many if-else statements when you have multiple options.

```
#!/bin/bash
```

```
echo "1) Add"
echo "2) Subtract"
echo "3) Multiply"
read -p "Choose option: " opt

read -p "Enter a: " a
read -p "Enter b: " b

case $opt in
  1) echo "sum = $((a + b))" ;;
  2) echo "sub = $((a - b))" ;;
  3) echo "mul = $((a * b))" ;;
  *) echo "Invalid option" ;;
esac
```

Explanation:

- Each option ends with `;;`
- `*)` is the default case (like else).
- **esac** ends the case block (case backwards).

13. Functions (Reuse Code)

Functions help you reuse code and keep scripts clean.

```
#!/bin/bash
```

```
greet() {  
    echo "Hello $1"  
}
```

```
greet "Ali"  
greet "Sara"
```

Explanation:

- **greet()** creates a function named greet.
- **\$1** inside a function means the first argument passed to it.

Return status (success/failure):

```
#!/bin/bash
```

```
is_even() {  
    n=$1  
    if [  $$(n \% 2)$  -eq 0 ]; then  
        return 0    # 0 means success/true  
    else  
        return 1    # non-zero means failure/false  
    fi  
}
```

```
read -p "Enter n: " n  
if is_even $n; then  
    echo "Even"  
else  
    echo "Odd"  
fi
```

14. Command-Line Arguments (\$1, \$2, ...)

Instead of asking input, a script can take values when you run it.

```
#!/bin/bash
# save as args.sh

echo "Script name: $0"
echo "First arg:   $1"
echo "Second arg:  $2"
```

Run it like this:

```
chmod +x args.sh
./args.sh Ali 20
```

You can also check number of arguments using \$#:

```
echo "Total args: $#"
```

15. Exit Status (\$?) and Simple Error Handling

Every Linux command returns an exit status. 0 means success, non-zero means failure.

```
#!/bin/bash
```

```
ls /does_not_exist  
echo "Exit status = $?"
```

Example: check if a command succeeded

```
#!/bin/bash
```

```
mkdir test_folder  
if [ $? -eq 0 ]  
then  
    echo "Folder created successfully"  
else  
    echo "Failed to create folder"  
fi
```

16. Practice Tasks (Homework / Lab)

Start from easy tasks and move to medium tasks.

Easy

- 1 Print your name, roll, and department using echo.
- 2 Take two numbers and print sum, sub, mul.
- 3 Check whether a number is even or odd.
- 4 Check whether age is eligible (≥ 18).
- 5 Print numbers 1 to 10 using a while loop.

Medium

- 1 Find the largest of 3 numbers (use if-elif-else).
- 2 Check if a file exists and is readable (-f and -r).
- 3 Create a menu using case: 1) date 2) whoami 3) pwd 4) exit.
- 4 Count how many files are in the current directory (hint: ls | wc -l).
- 5 Write a backup script: copy a folder to another name with date (advanced).

End of notes. You can add your own examples and classroom exercises.